Build a Full Airline Application System with MongoDB & Node.js

A comprehensive systems analysis project guiding you through every stage — from database design to deployment — culminating in a live demo.

DUE: APRIL 15, 2026

FULL STACK WEB APP

Course Overview & Project Goal

1

Build

Develop a web-based Airline Application using MongoDB and Node.js

2

Demo

Deliver a fully working demonstration by **April 15th, 2026**

3

Learn

Master backend API design, database modeling, frontend integration, and deployment

4

Grading

Evaluated on functionality, code quality, UI design, and successful deployment

CHAPTER 1

Understanding the Airline Application Domain

Before writing a single line of code, let's explore the core business domain your application will serve.

What is an Airline Application?

Flight Management

Manage flights, schedules, routes, and real-time availability data across multiple destinations.

Bookings & Passengers

Enable users to search flights, book seats, and manage their reservations — including cancellations.

Authentication & Roles

Secure login with role-based access: administrators manage the system, customers book flights.

Payments

Process booking payments, track transaction statuses, and handle refunds for canceled tickets.

Why MongoDB & Node.js?

MongoDB

A flexible document model that naturally fits complex, nested flight and booking data — no rigid table schemas required.

Node.js

An efficient, event-driven JavaScript backend capable of handling real-time updates, concurrent requests, and rapid API development.

Industry Standard

Both technologies are scalable, widely adopted, and supported by thriving open-source communities.

The Airline Booking Experience

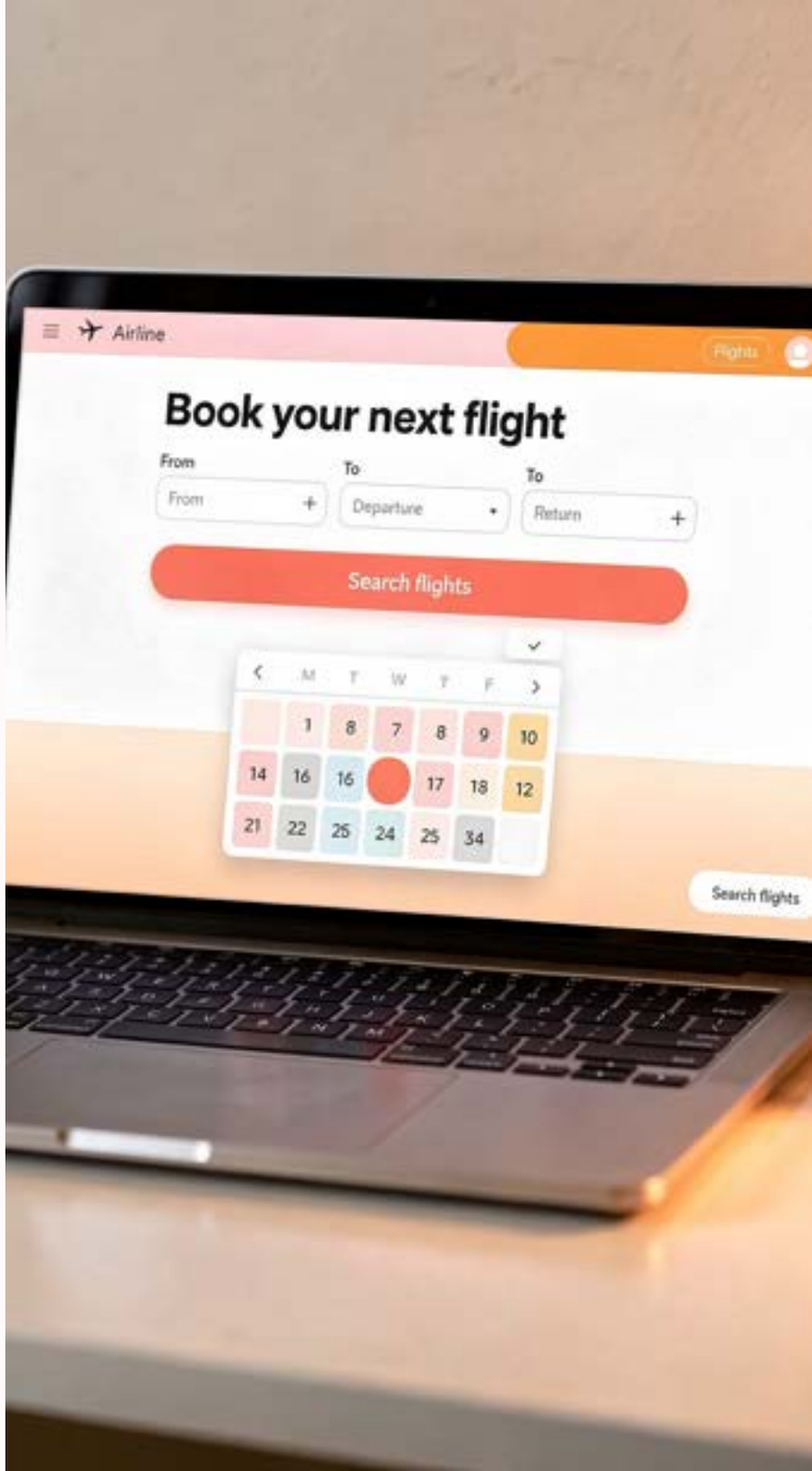
Your application will feature a clean, intuitive interface where users can search flights by origin, destination, and date — then seamlessly complete their booking in just a few clicks.

Search

Select

Pay

Confirm



CHAPTER 2

Project Setup & Tools

Everything you need to get your development environment running — all with free tools and platforms.

Development Environment Setup

01

Install Node.js & npm

Download from **nodejs.org**. Verify installation with `node -v` and `npm -v` in your terminal.

02

Create MongoDB Atlas Cluster

Sign up at MongoDB Atlas and create a free M0 cluster. Save your connection string for later use.

03

Choose a Code Editor

VS Code is recommended — install extensions for ESLint, Prettier, and MongoDB for VS Code.

04

Set Up Version Control

Create a GitHub repository. Initialize with `git init` and push your project structure immediately.

Free Online Development Platforms

You can build and host this entire application **at no cost** using these platforms:

MongoDB Atlas

Free M0 tier for your cloud database. Visit **mongodb.com/cloud/atlas**

Replit

Browser-based IDE with Node.js & MongoDB integration. Visit **replit.com**

Glitch

Quick Node.js hosting with live editing. Visit **glitch.com**

Project Structure Overview

Organize your codebase into a clean, modular folder structure. This separation of concerns makes your application maintainable and scalable.

Keep your `.env` file in `config/` and **never** commit it to GitHub. Add it to `.gitignore`.

```
airline-app/
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   ├── Flight.js
│   │   ├── User.js
│   │   └── Booking.js
│   ├── routes/
│   │   ├── flights.js
│   │   ├── bookings.js
│   │   └── users.js
│   └── server.js
├── frontend/
│   ├── index.html
│   ├── app.js
│   └── styles.css
├── .env
└── package.json
```

CHAPTER 3

Designing the Database Schema

A well-designed schema is the backbone of your application. Let's model the core entities for our airline system.

Key Entities & Collections

Flights

flightNumber, origin, destination, departureTime, arrivalTime, seatsAvailable, price

Users

username, email, passwordHash, role (admin or customer), createdAt

Bookings

userId, flightId, seatNumber, status (confirmed / canceled), bookingDate

Payments

bookingId, amount, paymentMethod, paymentStatus (pending / completed / refunded)

Sample MongoDB Flight Document

MongoDB stores data as JSON-like documents. Here is an example of a single flight record in your `flights` collection:

Notice how the document is **self-contained** — all relevant flight data lives in one place, making reads fast and efficient.

```
{
 "_id": "ObjectId('64a1b2c3...')",
 "flightNumber": "AA-1042",
 "origin": "New York (JFK)",
 "destination": "Los Angeles (LAX)",
 "departureTime": "2026-03-15T08:30:00Z",
 "arrivalTime": "2026-03-15T11:45:00Z",
 "seatsAvailable": 42,
 "totalSeats": 180,
 "price": 349.99,
 "status": "scheduled"
}
```

Entity Relationship Overview

Each user can create multiple bookings. Every booking references exactly one flight and may have an associated payment record. This forms the core relational structure of your application.

CHAPTER 4

Building the Mongoose Models

Mongoose provides a schema-based solution to model your application data, adding validation and structure on top of MongoDB's flexibility.

Flight Model – `models/Flight.js`

```
const mongoose = require('mongoose');

const flightSchema = new mongoose.Schema({
 flightNumber: { type: String, required: true, unique: true },
 origin: { type: String, required: true },
 destination: { type: String, required: true },
 departureTime:{ type: Date, required: true },
 arrivalTime: { type: Date, required: true },
 totalSeats: { type: Number, required: true },
 seatsAvailable:{ type: Number, required: true },
 price: { type: Number, required: true },
 status: { type: String, default: 'scheduled' }
});

module.exports = mongoose.model('Flight', flightSchema);
```

User Model – `models/User.js`

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  username: {
    type: String,
    required: true,
    unique: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  passwordHash: {
    type: String,
    required: true
  },
  role: {
    type: String,
    enum: ['admin', 'customer'],
    default: 'customer'
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports =
  mongoose.model('User', userSchema);
```

Key Design Decisions

Password Security

Store `passwordHash` only — never store plain text passwords. Use `bcryptjs` for hashing.

Role-Based Access

The `role` field with enum validation restricts values to `admin` or `customer`.

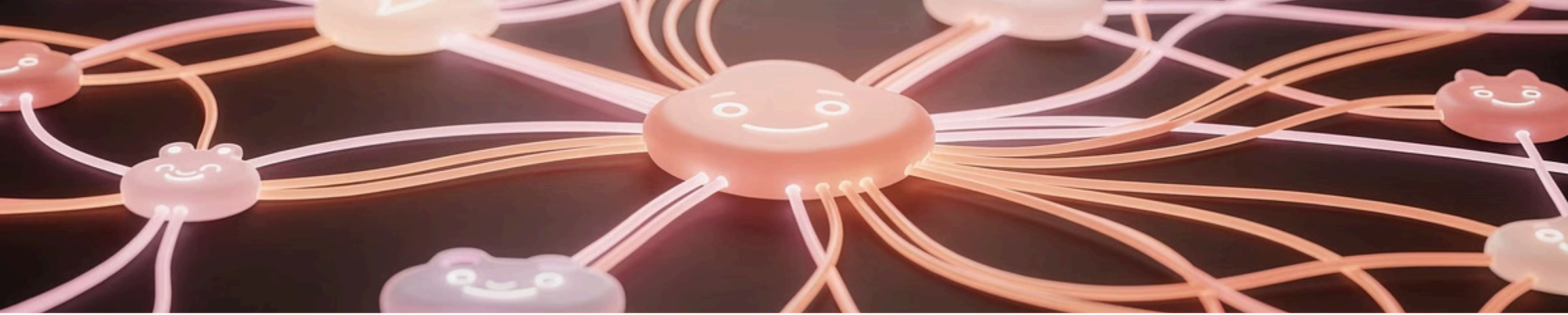
Unique Constraints

Both `username` and `email` are unique — preventing duplicate accounts.

Booking Model – `models/Booking.js`

```
const mongoose = require('mongoose');

const bookingSchema = new mongoose.Schema({
  userId:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  flightId: { type: mongoose.Schema.Types.ObjectId, ref: 'Flight', required: true },
  seatNumber: { type: String, required: true },
  status:     { type: String, enum: ['confirmed','canceled'], default: 'confirmed' },
  bookingDate:{ type: Date, default: Date.now }
});

module.exports = mongoose.model('Booking', bookingSchema);
```

Important: The `ref` property enables Mongoose `populate()` — letting you join User and Flight data into Booking queries automatically.



CHAPTER 5

Building the REST API

Your Express.js backend will expose RESTful endpoints for every operation in the airline system.

Server Setup – `server.js`

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Database Connection
mongoose.connect(process.env.MONGO_URI)
  .then(() =>
    console.log('MongoDB connected'))
  .catch(err =>
    console.error(err));

// Routes
app.use('/api/flights',
  require('./routes/flights'));
app.use('/api/users',
  require('./routes/users'));
app.use('/api/bookings',
  require('./routes/bookings'));

const PORT = process.env.PORT || 5000;
app.listen(PORT, () =>
  console.log(`Server on port ${PORT}`));
```

What This Does

This is the **entry point** of your application. It initializes Express, connects to MongoDB Atlas, registers middleware, and mounts your API routes.

Required Packages

```
npm install express mongoose
  cors dotenv bcryptjs
  jsonwebtoken
```

Environment File (`.env`)

```
MONGO_URI=mongodb+srv://user:pass
 @cluster.mongodb.net/airlineDB
PORT=5000
JWT_SECRET=your_secret_key
```

Flight Routes – routes/flights.js

```
const express = require('express');
const router = express.Router();
const Flight = require('../models/Flight');

// GET all flights
router.get('/', async (req, res) => {
 const flights = await Flight.find();
 res.json(flights);
});

// GET single flight
router.get('/:id', async (req, res) => {
 const flight = await Flight.findById(req.params.id);
 res.json(flight);
});

// POST create flight (admin only)
router.post('/', async (req, res) => {
 const flight = new Flight(req.body);
 await flight.save();
 res.status(201).json(flight);
});

// PUT update flight
router.put('/:id', async (req, res) => {
 const flight = await Flight.findByIdAndUpdate(req.params.id, req.body, { new: true });
 res.json(flight);
});

// DELETE flight
router.delete('/:id', async (req, res) => {
 await Flight.findByIdAndDelete(req.params.id);
 res.json({ message: 'Flight deleted' });
});

module.exports = router;
```

API Endpoints Summary

Method	Endpoint	Description
GET	/api/flights	Retrieve all available flights
POST	/api/flights	Create a new flight (admin only)
GET	/api/flights/:id	Get details for a specific flight
PUT	/api/flights/:id	Update flight info (admin only)
DELETE	/api/flights/:id	Remove a flight (admin only)
POST	/api/users/register	Register a new user account
POST	/api/users/login	Authenticate and receive JWT token
POST	/api/bookings	Create a new booking
GET	/api/bookings/user/:id	Get all bookings for a user
PUT	/api/bookings/:id/cancel	Cancel an existing booking

CHAPTER 6

User Authentication & Security

Secure your API with JSON Web Tokens (JWT) and password hashing using industry best practices.

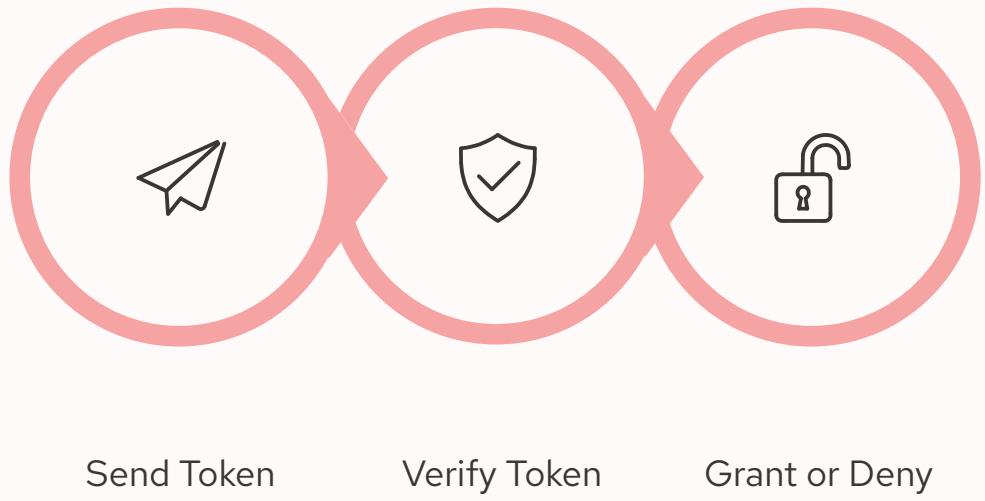
User Registration & Login Routes

```
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Register
router.post('/register', async (req, res) => {
 const { username, email, password, role } = req.body;
 const passwordHash = await bcrypt.hash(password, 10);
 const user = new User({ username, email, passwordHash, role });
 await user.save();
 res.status(201).json({ message: 'User registered successfully' });
});

// Login
router.post('/login', async (req, res) => {
 const { email, password } = req.body;
 const user = await User.findOne({ email });
 if (!user) return res.status(400).json({ error: 'User not found' });

 const isMatch = await bcrypt.compare(password, user.passwordHash);
 if (!isMatch) return res.status(400).json({ error: 'Invalid credentials' });

 const token = jwt.sign({ id: user._id, role: user.role },
 process.env.JWT_SECRET, { expiresIn: '24h' });
 res.json({ token, user: { id: user._id, username: user.username, role: user.role } });
});

module.exports = router;
```

JWT Authentication Middleware

```
// middleware/auth.js
const jwt =
require('jsonwebtoken');

const auth = (req, res, next) => {
  const token = req.header(
    'Authorization'
  )?.replace('Bearer ', '');

  if (!token) {
    return res.status(401)
      .json({ error: 'Access denied' });
  }

  try {
    const decoded = jwt.verify(
      token,
      process.env.JWT_SECRET
    );
    req.user = decoded;
    next();
  } catch (err) {
    res.status(400)
      .json({ error: 'Invalid token' });
  }
};

module.exports = auth;
```

How It Works



Securing Admin Routes

Role-Based Middleware

```
// middleware/adminOnly.js
const adminOnly = (req, res, next)
=> {
  if (req.user.role !== 'admin') {
    return res.status(403).json({
      error: 'Admin access required'
    });
  }
  next();
};

module.exports = adminOnly;
```

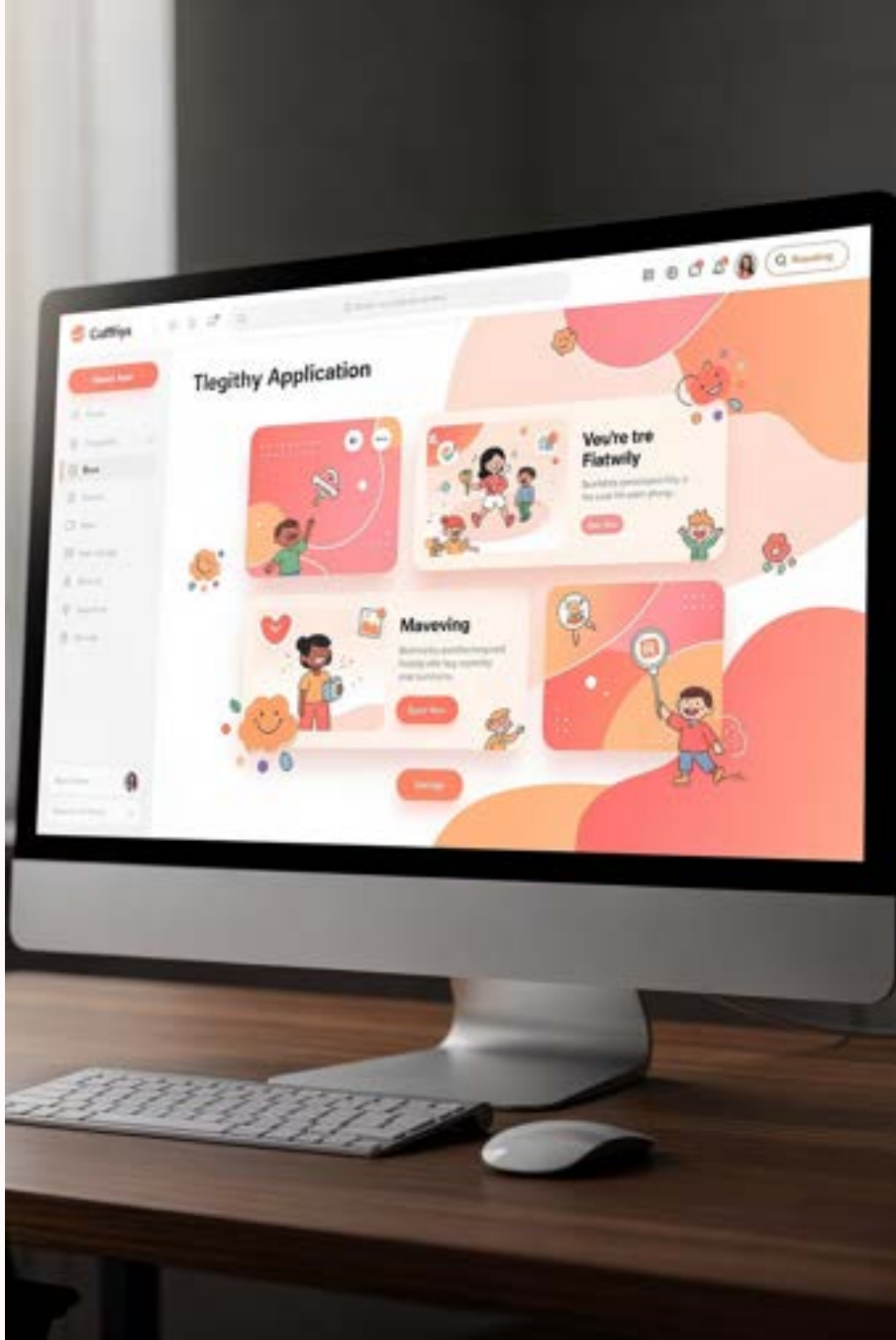
Apply to protected routes:

```
router.post('/', auth, adminOnly,
 async (req, res) => {
 // Only admins can create flights
});
```

CHAPTER 7

Building the Frontend

Create an intuitive, responsive web interface that connects to your Node.js API — giving users a seamless booking experience.



Frontend Architecture Options

Option A: React.js

Component-based UI with state management. Best for a polished, interactive single-page application.

Option B: Vanilla HTML/JS

Simple HTML, CSS, and vanilla JavaScript with `fetch()` API calls. Easier to start, great for beginners.

Option C: EJS Templates

Server-side rendered pages using Express view engine. Fast setup with dynamic HTML generation.

Sample Frontend – Flight Search Page

```
<!DOCTYPE html>
<html lang="en">
<head>
 <title>Airline App - Search Flights</title>
 <link rel="stylesheet" href="styles.css">
</head>
<body>
 <h1>Search Flights</h1>
 <form id="searchForm">
 <input type="text" id="origin" placeholder="From (e.g. JFK)">
 <input type="text" id="destination" placeholder="To (e.g. LAX)">
 <input type="date" id="date">
 <button type="submit">Search</button>
 </form>
 <div id="results"></div>
 <script src="app.js"></script>
</body>
</html>
```

Frontend JavaScript – API Integration

```
// app.js — Fetching flights from the backend API

document.getElementById('searchForm').addEventListener('submit', async (e) => {
 e.preventDefault();
 const origin = document.getElementById('origin').value;
 const destination = document.getElementById('destination').value;

 const response = await fetch(`/api/flights?origin=${origin}&destination=${destination}`);
 const flights = await response.json();

 const resultsDiv = document.getElementById('results');
 resultsDiv.innerHTML = flights.map(flight => `
 <div class="flight-card">
 <h3>${flight.flightNumber}</h3>
 <p>${flight.origin} → ${flight.destination}</p>
 <p>Departure: ${new Date(flight.departureTime).toLocaleString()}</p>
 <p>Price: $${flight.price} | Seats: ${flight.seatsAvailable}</p>
 <button onclick="bookFlight('${flight._id}')">Book Now</button>
 </div>
 `).join('');
});
```

Booking Flow – Complete Code

```
// Booking function in app.js
async function bookFlight(flightId) {
  const token = localStorage
    .getItem('token');

  if (!token) {
    alert('Please login first');
    window.location = '/login.html';
    return;
  }

  const seatNumber = prompt(
    'Enter seat (e.g. 12A):'
  );

  const response = await fetch(
    '/api/bookings', {
    method: 'POST',
    headers: {
      'Content-Type':
        'application/json',
      'Authorization':
        `Bearer ${token}`
    },
    body: JSON.stringify({
      flightId,
      seatNumber
    })
  });

  const data = await response.json();

  if (response.ok) {
    alert('Booking confirmed!');
  } else {
    alert(data.error);
  }
}
```

Booking API Route

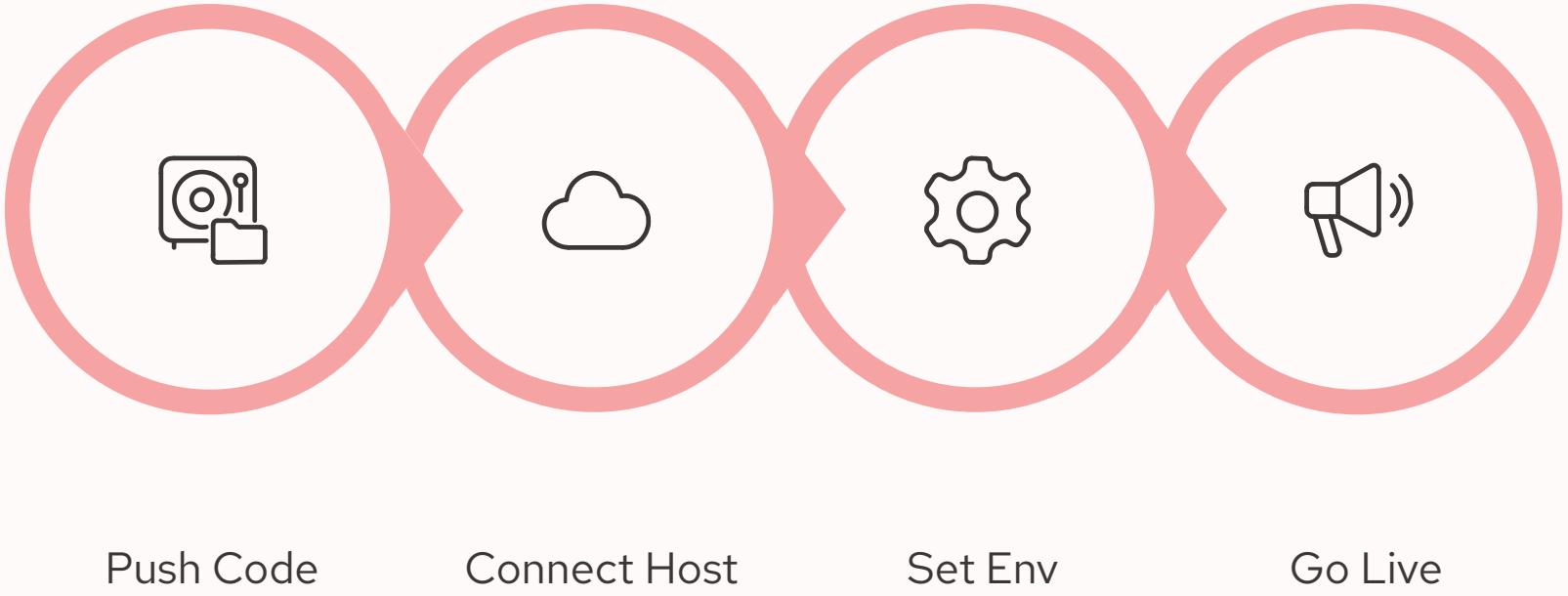
```
// routes/bookings.js
const auth = require(
 '../middleware/auth');

router.post('/', auth,
 async (req, res) => {
 const { flightId, seatNumber }
 = req.body;

 // Check seat availability
 const flight = await
 Flight.findById(flightId);
 if (flight.seatsAvailable < 1) {
 return res.status(400).json({
 error: 'No seats available'
 });
 }

 // Create booking
 const booking = new Booking({
 userId: req.user.id,
 flightId,
 seatNumber,
 status: 'confirmed'
 });
 await booking.save();

 // Update seat count
 flight.seatsAvailable -= 1;
 await flight.save();

 res.status(201).json(booking);
});
```

CHAPTER 8

Deployment & Going Live

Deploy your application to the cloud so it's accessible from anywhere — ready for your April 15th demo.

Deployment Workflow



Free hosting platforms like Render, Railway, and Glitch automatically pull from your GitHub repository and deploy your Node.js application within minutes.

Free Hosting Platforms for Deployment

Render

Free web service tier with automatic deploys from GitHub. Visit **render.com**

Railway

Simple deployment with $5 free monthly credit. Visit **railway.app**

Vercel

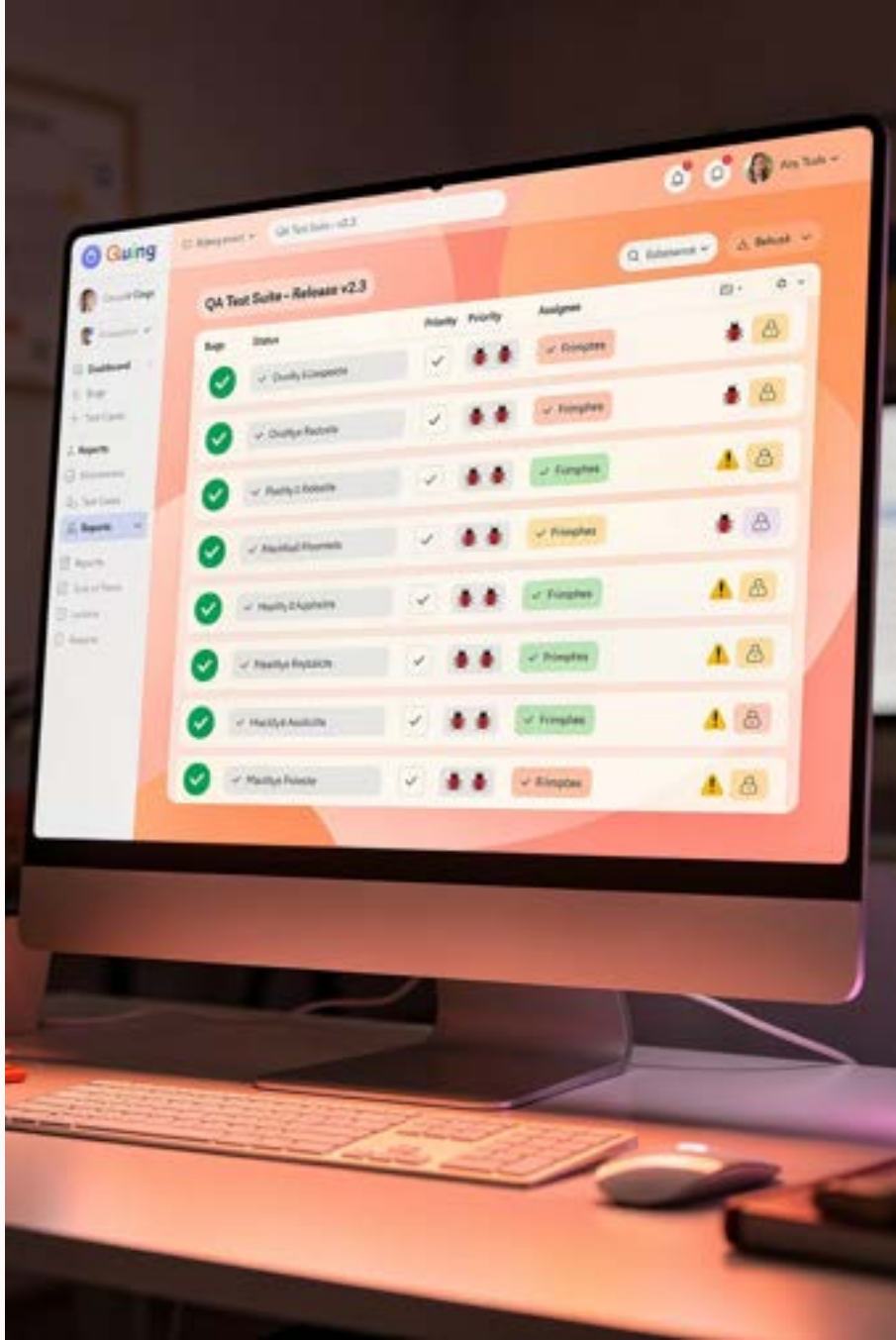
Great for frontend + serverless functions. Visit **vercel.com**

Reminder: Your MongoDB Atlas database is already cloud-hosted. You only need to deploy the Node.js server and static frontend files.

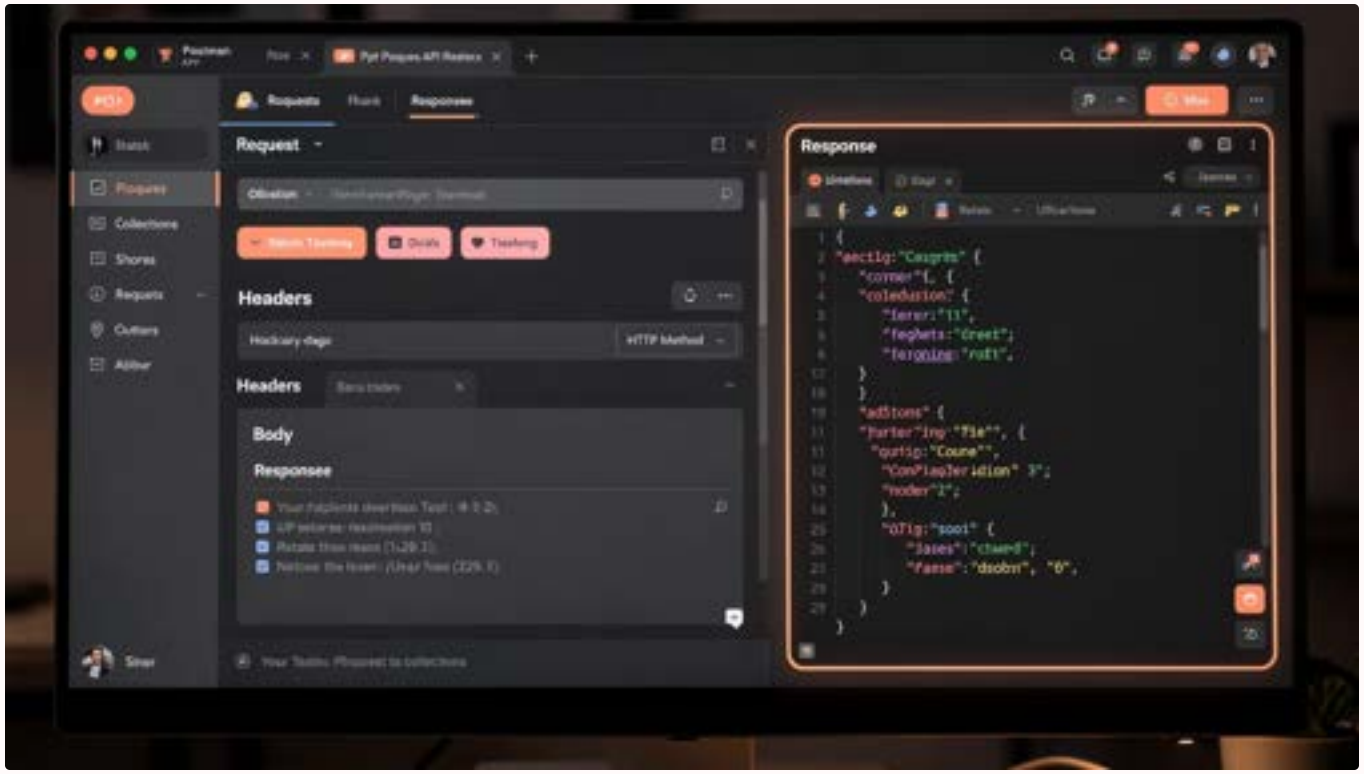
CHAPTER 9

Testing & Quality Assurance

Thorough testing ensures your application is reliable, secure, and ready for a flawless demo presentation.



Testing Your API with Postman



Manual API Testing

Use **Postman** (free) to test every endpoint before building the frontend.

- ### Test CRUD Operations

  Create, read, update, and delete flights, users, bookings

- ### Verify Auth Flow

  Register → Login → Use token for protected routes

- ### Check Edge Cases

  Invalid data, duplicate bookings, overbooking attempts

Testing Checklist

Database

Verify MongoDB connection, schema validation, data integrity

API

Test all endpoints, error handling, status codes, response formats

Frontend

Cross-browser testing, responsive design, form validation

Security

Auth flows, role restrictions, input sanitization, token expiry

CHAPTER 10

Demo Day Preparation

Your demo on **April 15th, 2026** is your chance to showcase everything you've built. Here's how to make it shine.

Demo Day Requirements

1 **Live Application**

Your app must be deployed and accessible via a public URL. No localhost demos allowed.

2 **Full Walkthrough**

Demonstrate user registration, login, flight search, booking, and admin management features live.

3 **Code Review**

Be prepared to explain your code structure, database schema decisions, and API design choices.

4 **Q&A**

Respond to questions about your implementation, trade-offs, and potential improvements.

Demo Script – What to Show

1. Register

Create a new customer account

2. Login

Authenticate and receive token

3. Search

Find flights by route and date

4. Book

Reserve a seat on a flight

5. Admin

Add/edit flights as admin

Pro Tip: Pre-load your database with sample flights and a test admin account before the demo so you don't waste time on setup.

CHAPTER 11

Grading Rubric & Point Breakdown

Understand exactly how your project will be evaluated. Every point counts — plan your work accordingly.

Grading Overview – Total: 100 Points

- Database Design
- Backend API
- Frontend UI
- Authentication
- Deployment
- Demo & Presentation

Grade Scale

The project is graded on a 100-point scale, divided across six major categories. Each category evaluates a critical aspect of your full-stack application.

90–100: A (Excellent)
80–89: B (Good)
70–79: C (Satisfactory)
60–69: D (Needs Improvement)
Below 60: F (Failing)

Category 1: Database Design – 20 Points

Criteria	Points	Evaluation
Mongoose schemas with proper data types and validation	5	Schema correctness
Appropriate relationships using ObjectId references	5	Data modeling
MongoDB Atlas setup and connection	4	Cloud database
Sample data seeding script	3	Test data quality
Indexing for performance optimization	3	Query efficiency

Category 2: Backend API – 25 Points

Criteria	Points	Evaluation
Complete CRUD operations for Flights	6	Full REST compliance
Complete CRUD operations for Bookings	6	Booking lifecycle
User registration and login endpoints	5	Auth endpoints
Proper error handling and status codes	4	Robustness
Clean, modular route organization	4	Code quality

Category 3: Frontend UI – 15 Points

Flight Search Interface – 4 pts

Functional search form with origin, destination, and date fields that returns matching results.

Booking Interface – 4 pts

Users can select a flight, choose a seat, and complete the booking through an intuitive flow.

Responsive Design – 4 pts

Application renders correctly on desktop and mobile screen sizes with clean CSS styling.

User Dashboard – 3 pts

Logged-in users can view their bookings and cancel reservations from a personal dashboard.

Categories 4–6: Authentication, Deployment & Demo

Authentication – 15 pts

JWT implementation (5), password hashing with bcrypt (5), role-based access control for admin vs. customer (5)

Deployment – 10 pts

Live URL on Render/Railway (4), environment variables secured (3), GitHub repo with README (3)

Demo & Presentation – 15 pts

Live walkthrough (5), code explanation (5), Q&A responses (5)

Complete Points Summary

20

Database

Schema, relationships, Atlas setup

25

Backend API

CRUD, routes, error handling

15

Frontend

UI, search, booking, responsive

15

Auth

JWT, bcrypt, role control

10

Deploy

Live URL, env vars, GitHub

15

Demo

Walkthrough, code review, Q&A

Bonus Points – Extra Credit Opportunities

→ **Payment Integration (+5 pts)**

Integrate Stripe test mode for simulated payment processing on bookings.

→ **Email Notifications (+3 pts)**

Send booking confirmation emails using Nodemailer or SendGrid free tier.

→ **Seat Map Visualization (+5 pts)**

Build an interactive seat selection UI showing available and occupied seats.

→ **Unit Testing (+2 pts)**

Write automated tests using Jest or Mocha for your API endpoints.

Project Timeline & Milestones

Week 1–2: Setup

Environment, MongoDB Atlas, project structure, GitHub repo

Week 3–4: Models & API

Mongoose schemas, all CRUD routes, authentication middleware

Week 5–6: Frontend

Search page, booking flow, user dashboard, responsive CSS

Week 7: Integration

Connect frontend to API, end-to-end testing, bug fixes

Week 8: Deploy & Demo Prep

Deploy to Render, seed data, rehearse demo, final polish

April 15, 2026

Demo Day! Present your live application

Clear Skies Ahead

You Have Everything You Need

Follow the code, use the free platforms, hit the milestones — and you'll deliver an impressive airline application on demo day.

Due Date

April 15, 2026

Total Points

100 + Bonus

Go Build

Start today!